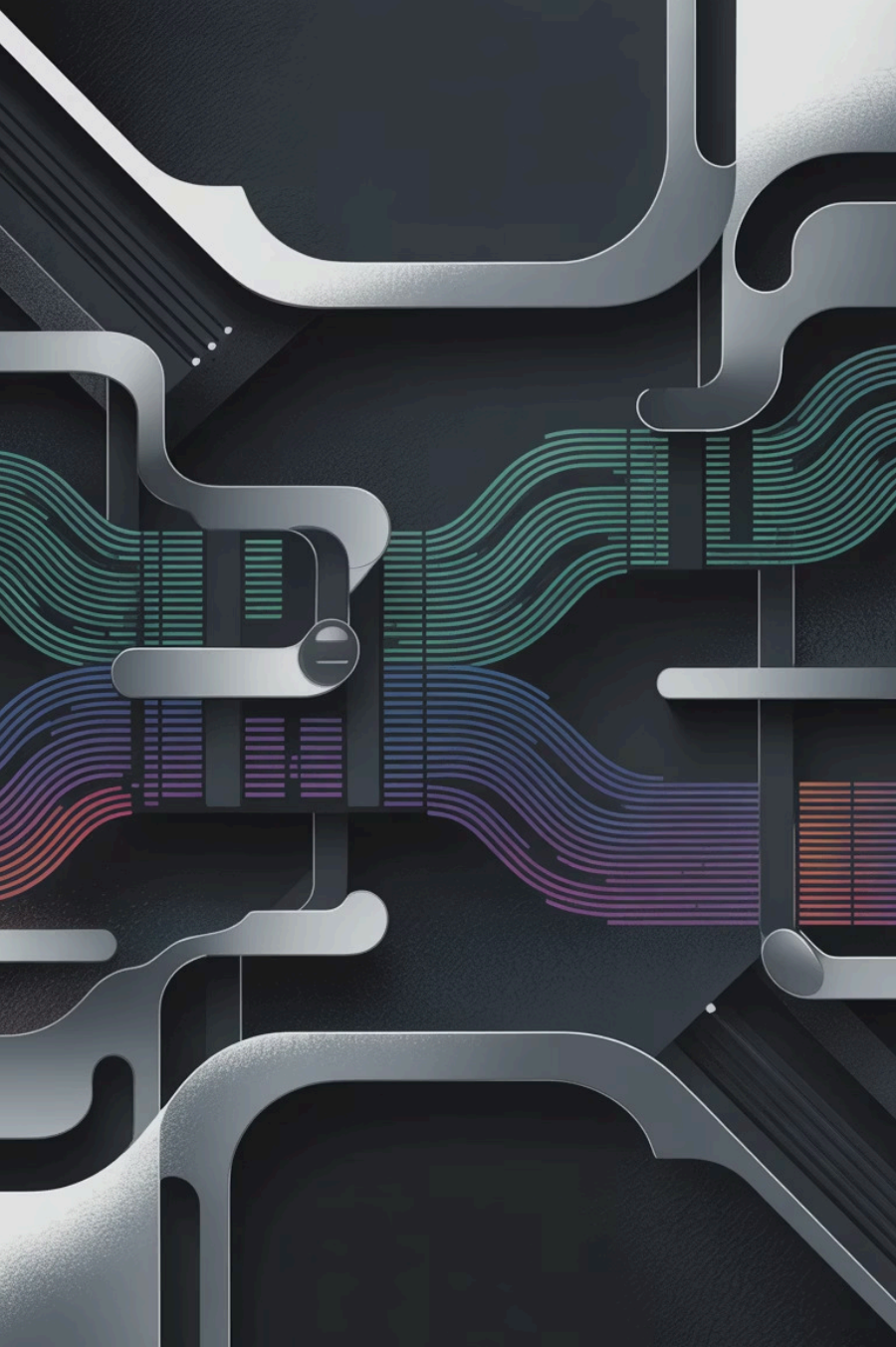




Module 04 - Prompt Engineering & Task-to-Prompt Mapping

Building the Foundation for AI Engineering Excellence

Today's Learning Journey

01

Role Prompting Fundamentals

Define system personas and expert behaviors to shape model outputs

03

Step Lists & Reasoning

Structure multi-step instructions for complex reasoning tasks

05

Style & Tone Control

Fine-tune communication style for target audiences and contexts

02

Constraint Engineering

Apply boundaries, formatting rules, and output specifications

04

Acceptance Testing

Define validation criteria and quality gates within prompts

06

Hands-On Lab Practice

Apply techniques to real-world sentiment analysis and classification

The Power of Instruction-Based Prompting

What Makes It Essential?

Instruction-based prompting transforms vague requests into precise, repeatable operations. Unlike simple queries, structured instructions establish clear expectations, define output formats, and embed quality controls directly into the prompt itself.

This approach enables LLMs to consistently deliver production-grade results across diverse tasks: sentiment analysis, entity extraction, document summarization, and content generation.



Anatomy of a Production Prompt

System Role

Defines the AI's expertise, perspective, and behavioral foundation

Task Context

Provides background information and situational awareness

Instructions

Specifies exact operations, sequences, and processing logic

Constraints

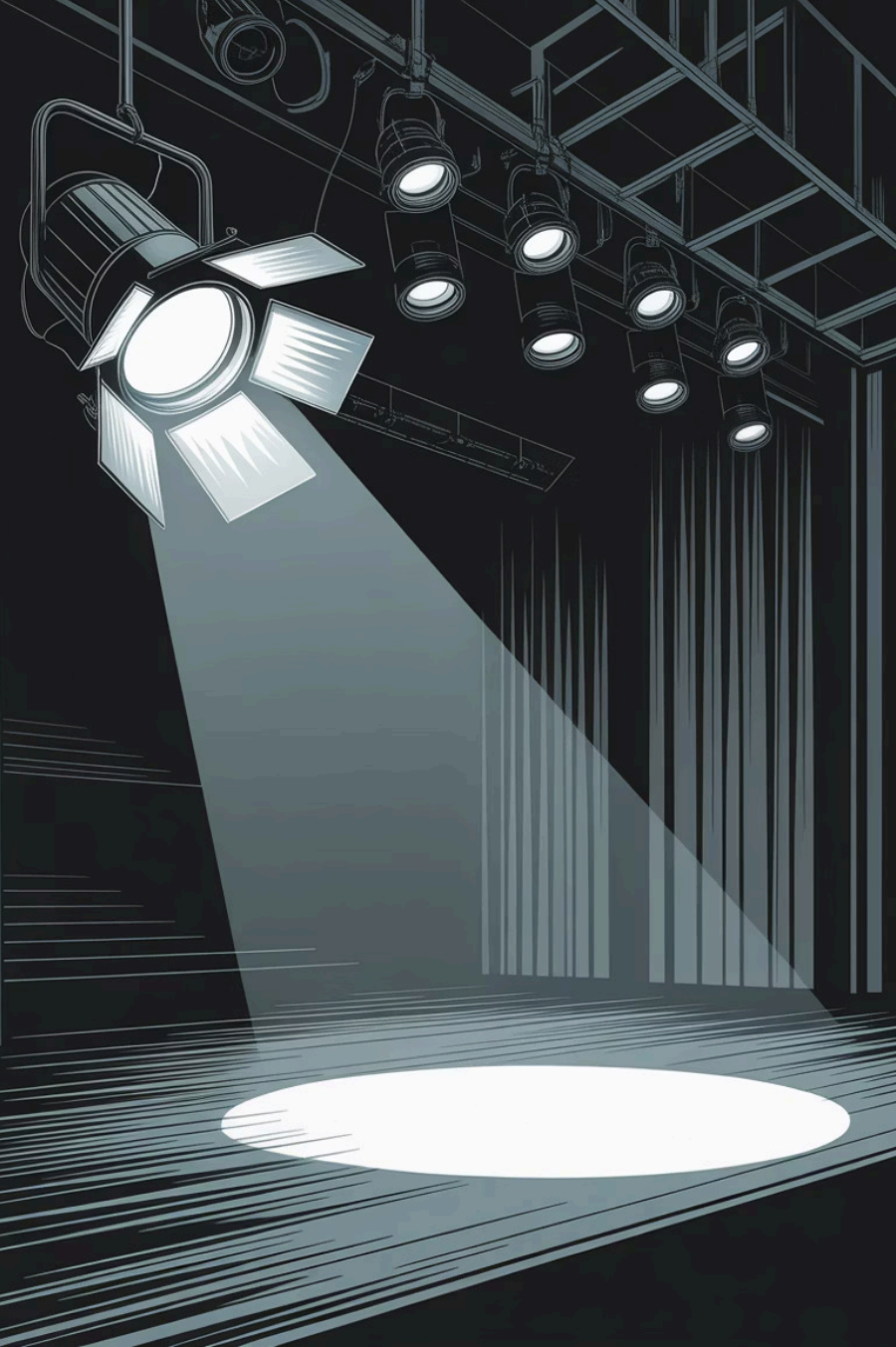
Establishes boundaries, format requirements, and limitations

Examples

Demonstrates desired input-output patterns when needed

Validation

Defines quality criteria and acceptance tests



Role Prompting: Setting the Stage

Role prompting establishes the AI's persona, expertise level, and behavioral patterns. By defining a specific role, you activate relevant knowledge domains and set appropriate communication styles. This technique significantly improves output quality by grounding the model in a consistent perspective.

Effective roles combine expertise (what the AI knows), perspective (how it thinks), and audience awareness (how it communicates). The role becomes the lens through which all subsequent instructions are interpreted.

Role Definition: From Vague to Precise

Weak Role Definition

You are **a** helpful assistant.

This generic role provides no expertise context, behavioral guidance, or quality expectations. The model lacks domain grounding.

Strong Role Definition

You **are** an NLP engineer analyzing customer-feedback sentiment. Pay **close** attention **to** emotion, sarcasm, **and** context-dependent meaning; prefer calibrated judgments **over** confident guesses.

This role activates domain knowledge, establishes expertise depth, and sets behavioral expectations for nuanced analysis.

Role Specialization Strategies

Domain Expert Roles

Use case: Technical analysis, specialized knowledge tasks

Example: "You are a senior data scientist specializing in time-series anomaly detection..."

Process-Oriented Roles

Use case: Workflow execution, multi-step operations

Example: "You are a meticulous document processor following ISO 9001 quality standards..."

Analytical Roles

Use case: Critical thinking, evaluation, reasoning

Example: "You are a critical reviewer evaluating research paper methodologies..."

Creative Roles

Use case: Content generation, ideation, brainstorming

Example: "You are an innovative prompt engineer designing novel few-shot examples..."

Constraints: The Guardrails of Quality



Constraints define the boundaries within which the model operates. They prevent hallucinations, ensure format compliance, control output length, and enforce business rules. Without constraints, even well-instructed models can produce unusable outputs.

Effective constraints are specific, testable, and enforceable. They should address format requirements, content boundaries, length limits, and prohibited behaviors. Think of constraints as the contract between your application and the LLM.

Essential Constraint Categories



Format Constraints

JSON schema, CSV structure, markdown formatting, XML tags

Output must be valid JSON with keys: sentiment, confidence, reasoning



Length Constraints

Token limits, character counts, sentence restrictions

Response must be 50-100 words maximum



Content Constraints

Topic boundaries, vocabulary restrictions, prohibited content

Do not include personally identifiable information (PII)



Behavioral Constraints

Response style, certainty levels, citation requirements

If uncertain, respond with "INSUFFICIENT_DATA" rather than guessing

Constraint Implementation: JSON Schema Example

For structured outputs like sentiment analysis, define exact JSON schemas within your prompts. This ensures consistent parsing and integration with downstream systems.

Constraint: Output Format

Return your analysis **as** valid JSON matching this schema:

```
{  
  "sentiment": "positive" | "negative" | "neutral",  
  "confidence": float (0.0 to 1.0),  
  "key_phrases": array of 1-5 strings,  
  "reasoning": string (one concise sentence; ~15-20 words),  
  "detected_emotion": string or null  
}
```

Do not **include** any text outside the JSON **object**.

Do not **use** markdown code fences.

Ensure all strings are properly escaped.

Length is a soft target — the model can't count characters. Enforce any hard limit in code (reasoning[:N]).

Constraint Implementation: JSON Schema Example

⚠ **CURRENCY — Structured Outputs (2025+)** Prompt-only JSON instructions (below) are the provider-agnostic *floor*. In production, prefer **provider-enforced structured outputs**, which compile your JSON Schema into a decode constraint — the model *cannot* emit non-conforming output. • **OpenAI:** `response_format={"type":"json_schema","json_schema":{"strict":true,"schema":{"...}}}` • **Anthropic:** beta header `anthropic-beta: structured-outputs-2025-11-13` + `output_format` (JSON Schema), or `messages.parse()` with a Pydantic model. Both guarantee the *shape*, not the *correctness* — you can still get a well-formed wrong answer, so keep semantic validation.

```
{
  "type": "object",
  "properties": {
    "sentiment": {"type": "string", "enum": ["positive", "negative", "neutral"]},
    "confidence": {"type": "number", "minimum": 0.0, "maximum": 1.0},
    "key_phrases": {"type": "array", "items": {"type": "string"}, "minItems": 1, "maxItems": 5},
    "reasoning": {"type": "string"},
    "detected_emotion": {"type": ["string", "null"]}
  },
  "required": ["sentiment", "confidence", "key_phrases", "reasoning", "detected_emotion"],
  "additionalProperties": false
}
```

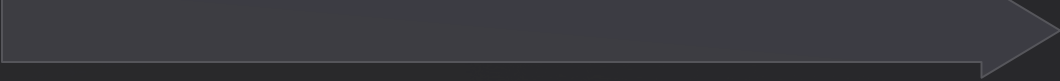
Step Lists: Breaking Down Complexity

Complex tasks require decomposition. Step lists transform ambiguous instructions into sequential, executable operations. Each step should be atomic, testable, and ordered. This technique is especially powerful for reasoning tasks, multi-stage analysis, and quality-critical operations.

Well-designed step lists improve accuracy, enable debugging, and make model reasoning transparent. They serve as both instruction and documentation.

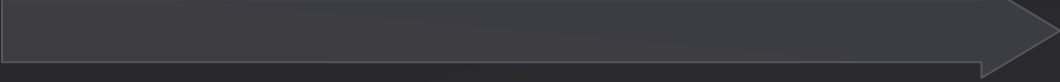


Step List Design Principles



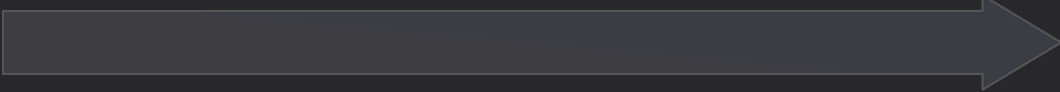
Make Steps Atomic

Each step should accomplish one clear subtask that can be verified independently



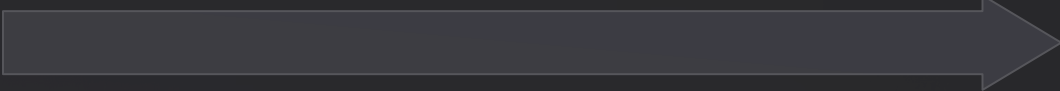
Include Decision Points

Specify conditional logic and branching criteria when appropriate



Sequence Logically

Order steps from input processing through analysis to output formatting



Define Success Criteria

Clarify what "complete" means for each step before moving forward

Example: Sentiment Analysis Step List

Here's a production-grade step list for analyzing customer feedback sentiment:

Follow these steps in order:

1. Read the input text and identify the primary subject
2. Extract key phrases that indicate emotional tone
3. Classify sentiment as positive, negative, or neutral based on:
 - Explicit sentiment words (great, terrible, okay)
 - Implicit sentiment (sarcasm, context clues)
 - Overall message intent
4. Assign confidence score (0.0-1.0) based on:
 - Clarity of sentiment signals
 - Presence of mixed emotions
 - Text length and completeness
5. Identify the dominant emotion (joy, anger, frustration, etc.)
6. Write a brief reasoning statement explaining your classification
7. Format output as JSON per the schema provided
8. Re-read your JSON for obvious structural errors before returning (final parse-validation is performed in code)

Step Lists with Conditional Logic

Advanced: Branching Instructions

1. Analyze input length
 - If < 10 words: flag **as** TOO_SHORT
 - If > 500 words: extract key sentences first
 - Otherwise: proceed to step 2
2. Detect language
 - If non-English: **return** UNSUPPORTED_LANGUAGE
 - Otherwise: **continue**
3. Check **for** sarcasm markers
 - If detected: the wording may mean the opposite of what it says. Look **for** stronger context before deciding, and lower confidence **if** the meaning **is** still unclear.
4. Generate **final** classification

Conditional steps handle edge cases, improve robustness, and prevent invalid outputs. They make your prompts resilient to real-world input variability.

This approach is essential for production systems where input quality varies significantly.



Acceptance Tests: Built-In Quality Assurance

Acceptance tests are validation criteria embedded directly in your prompts. They instruct the model to self-check outputs against defined quality standards before returning results. This technique reduces post-processing work and catches errors at generation time.

Think of acceptance tests as unit tests for your LLM outputs. They specify exactly what "correct" looks like and prevent invalid responses from reaching your application layer.

Types of Acceptance Tests

1

Format Validation

Verify output structure matches schema

Example: "Confirm JSON is valid and contains all required keys"

2

Logical Consistency

Check for internal contradictions

Example: "Ensure reasoning aligns with classification"

3

Boundary Compliance

Validate against constraints

Example: "Verify confidence is between 0.0 and 1.0"

4

Completeness Checks

Confirm all required fields populated

Example: "All array fields must have at least one element"

Implementing Acceptance Tests in Prompts

Add a validation section to your prompt that runs before final output:

Before returning your response, verify:

- ✓ Output looks like well-formed JSON (self-check; final validation happens in code)
- ✓ "sentiment" field contains only: positive, negative, or neutral
- ✓ "confidence" is a float between 0.0 and 1.0
- ✓ "key_phrases" has 1-5 items
- ✓ "reasoning" is a single concise sentence
- ✓ All string values are properly quoted and escaped

If any check fails, revise your output before returning.

Note: these are self-checks that reduce errors — your application still validates the returned JSON programmatically (e.g., json.loads / Pydantic) and retries on failure.

This self-checking mechanism dramatically reduces downstream errors and improves data quality.

Style and Tone Controls

Style and tone controls adjust how the model communicates while preserving technical accuracy. These controls are crucial when outputs face different audiences: technical teams need precision and jargon, while business stakeholders require clarity and accessibility.

Effective tone controls specify vocabulary level, sentence complexity, use of technical terms, formality, and emotional register. They transform the same underlying analysis into audience-appropriate communication.



Style Control Dimensions



Tone Specification Examples

Technical Tone

"Use precise ML terminology. Include confidence intervals, F1 scores, and statistical significance where relevant. Write for data scientists."

Business Tone

"Explain findings in business terms. Avoid jargon. Focus on actionable insights and ROI. Write for executives."

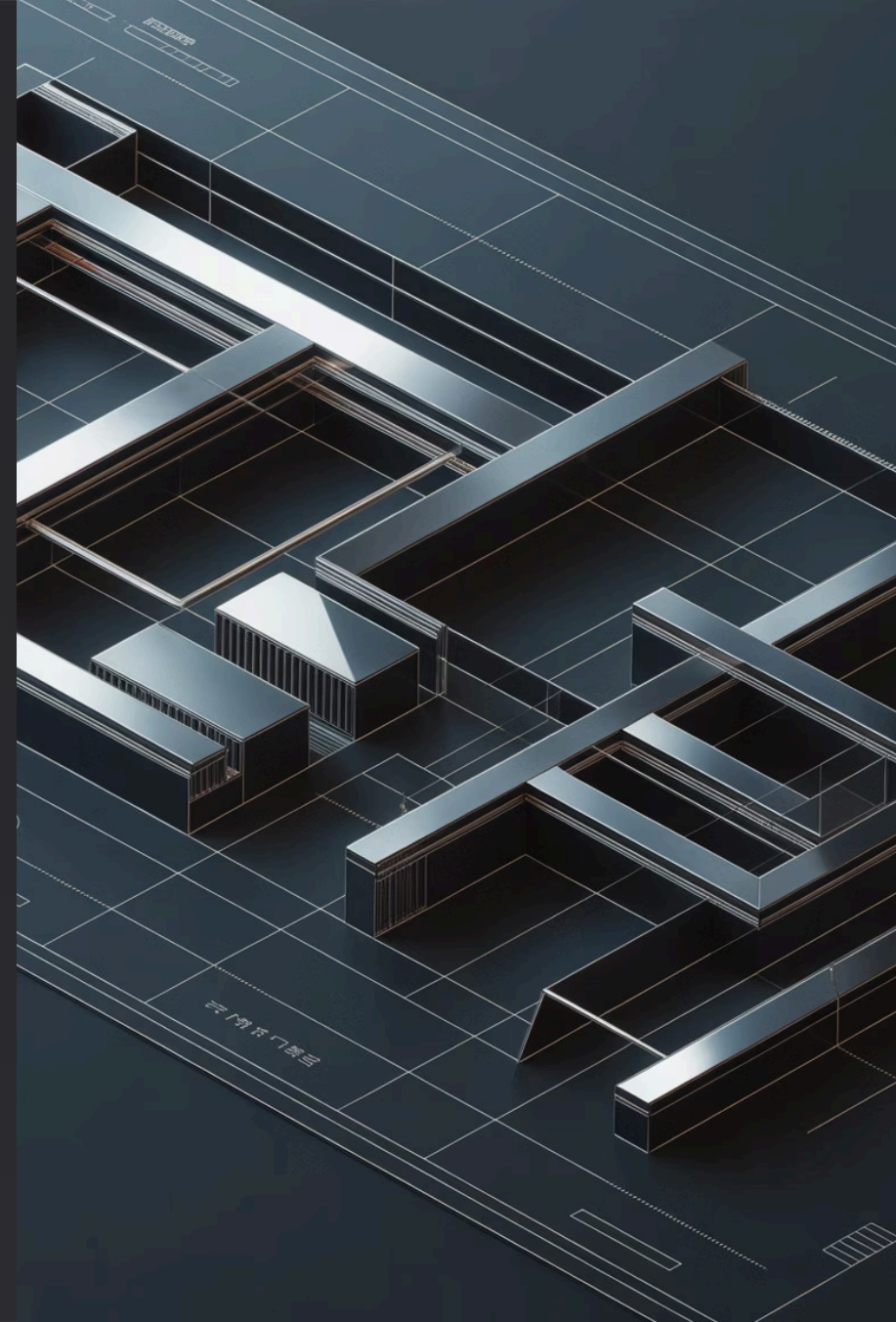
Educational Tone

"Explain concepts clearly with examples. Define technical terms. Build understanding progressively. Write for learners."

Putting It All Together: Complete Prompt Architecture

A production-grade prompt integrates all five components: role, constraints, step lists, acceptance tests, and style controls. This architecture creates a robust, maintainable, and performant prompt that delivers consistent results across diverse inputs.

The following example demonstrates how these elements work together in a real sentiment analysis prompt for customer feedback processing.



Production Prompt Template

ROLE

You are an NLP engineer analyzing customer-feedback sentiment. Attend to emotional nuance, sarcasm, and context-dependent meaning.

TASK

Analyze the sentiment of customer feedback text.

CONSTRAINTS

- Output valid JSON only (no markdown, no explanations)
- Sentiment must be: positive, negative, or neutral
- Confidence must be float 0.0-1.0
- *key_phrases must have 1-5 items*
- *reasoning must be one concise sentence (~15-20 words)*
- *detected_emotion* must be a string, or null if none is dominant

STEPS

1. Read input and identify primary subject
2. Extract sentiment-indicating phrases
3. Classify sentiment based on explicit and implicit signals
4. Assign confidence based on signal clarity
5. Identify dominant emotion if present
6. Format as JSON per schema
7. Validate output before returning

ACCEPTANCE TESTS

Before returning, verify JSON is valid and all fields are populated.

STYLE

Technical precision. Concise reasoning. No hedging language.



Advanced Pattern: Few-Shot Examples Integration

While today focuses on instruction-based techniques, few-shot examples complement these approaches. You can combine explicit instructions with demonstrations to handle edge cases that are easier to show than describe.

Example: After your step list, include 2-3 representative input-output pairs showing how to handle sarcasm, mixed sentiment, or short reviews. This hybrid approach leverages both instruction clarity and pattern learning.

Production Considerations

Version Control

Track prompt versions in Git. Use semantic versioning. Document changes and performance impacts.

Monitoring

Log prompt versions with outputs. Track accuracy drift. Set up alerts for format failures.

Testing

Maintain test suites with edge cases. Run regression tests on prompt changes. Measure before deploying.

Cost Optimization

Balance prompt length vs. accuracy. Test shorter instruction variants. Monitor token usage.

Tools for Prompt Experimentation



In upcoming sessions, we'll explore **PromptLayer** and **LangSmith** for systematic prompt development. These platforms provide version control, A/B testing frameworks, performance analytics, and collaboration features.

They transform ad-hoc prompt iteration into engineering discipline with metrics, reproducibility, and team workflows.

Key Takeaways



Structure Drives Quality

Role prompting, constraints, and step lists transform vague requests into precise, repeatable operations that deliver production-grade outputs.



Style Adapts to Audience

Tone controls make the same analysis accessible to different stakeholders without sacrificing technical accuracy.



Validate Early and Often

Acceptance tests embedded in prompts catch errors at generation time, reducing post-processing and improving reliability.



Measure Everything

Track accuracy, consistency, and format compliance across prompt iterations to demonstrate value and justify design choices.



Thanks